

# Die Anwendung zellulärer Automaten in der Bildverarbeitung – Analyse und Vergleich

Ihrenberger Julia  
Fakultät für Informatik  
Technische Hochschule Rosenheim  
Rosenheim, Deutschland  
julia.ihrenberger@stud.th-rosenheim.de

**Abstract**— Bildrauschen, insbesondere sogenanntes Salz und Pfeffer Rauschen (engl.: *salt-and-pepper noise*), ist ein weitverbreitetes Problem in Bereichen der Fotografie, Medizin und Datenverarbeitung. Ein grundlegendes Verfahren zur Entfernung von solchen Störgeräuschen stellt der Medianfilter dar, welcher besonders effektiv bei der Entfernung von Salz und Pfeffer Rauschen ist, jedoch einen hohen Rechenaufwand birgt. Darüber hinaus lässt die Leistung des Filters bei höheren Rauschdichten rapide ab. Im Forschungsbereich der Bildverarbeitung wurden unzähligen Filtermethoden entwickelt, welche diese Verfahren auch für höhere Dichten aufbessert oder neue Ansätze, die diesen vollständig übertreffen. Ein Ansatz für solche Verfahren ist in der Anwendung von Rauschfiltern basierend auf zellulären Automaten zu finden, was in den letzten Jahrzehnten stetig an Popularität gewonnen hat. Das emergente Verhalten sowie der niedrige Rechenaufwand von zellulären Automaten bietet großes Potential zur Findung von einem kostengünstigen Alternativverfahren zum Medianfilter. In dieser Arbeit werden vier solcher Filteralgorithmen präsentiert, die Konzepte und Charakteristika von zellulären Automaten umsetzen. Hierfür finden mehrere Vergleiche mit dem Medianfilter sowie der Algorithmen untereinander statt. Desweiteren folgt eine Analyse der Auswirkung von zellulären Automaten auf die Verfahren. Der Vergleich hat ergeben, dass alle vier Algorithmen eine höhere Effektivität bei jeglichen Rauschdichten erreichen.

**Keywords**— *cellular automata, median filter, noise filtration, salt-and-pepper noise, structural similarity index, peak signal to noise ratio*

## I. EINLEITUNG

Das Bereinigen und Entfernen von Rauschen in digitalen Bildern ist eine grundlegende Problemstellung in Bereichen der Fotografie, Medizin und Datenverarbeitung. Besonders in den zuletzt genannten Bereichen führt es zur ungewollten Addition oder Verzerrung von Information, welche Ergebnisse und Auswertungen verfälschen oder anderweitig beeinträchtigen können.

Es gibt viele verschiedene Arten von Bildrauschen, wobei das sogenannte Salz und Pfeffer Rauschen in Graubildern besonders weit verbreitet ist [1]. Ursache von Salz und Pfeffer Rauschen stellen beispielsweise defekte Kamerasensoren, Übertragungsfehler oder Bildfehler bei Aufbewahrung und Abruf dar [1].

Um derartigen Bildfehlern entgegenzuwirken, gibt es eine große Bandbreite an Filtertechniken, welche durch die Manipulation der Pixelwerte die Qualität des betroffenen Bilds restaurieren oder anderweitig aufbessern.

Weit verbreitete Herangehensweise zu dieser Problemstellung stellen Filtermethoden da, die auf den sogenannten

Medianfilter basieren, welche besonders effektiv gegen SPN sind.

Der Medianfilter weist jedoch nur begrenzte Effektivität bei besonders hohen Rauschdichten auf, weshalb es viele Ansätze und wissenschaftliche Arbeiten gibt, welche eine direkte Besserung des Medianfilters als Ziel vorzeigen.

Eine kleine Untergruppe von diesem Forschungsbereich stellen Filterverfahren dar, welche auf den Prinzipien von zellulären Automaten (CA, engl.: *cellular automata*) basieren. In mehreren Werken [2] [3] [4] zeigt CA großes Potential in jeglichen Disziplinen der Bilderverarbeitung wie das Bereinigen von Bildrauschen, Identifikation von Bildkanten und die Berechnung von konvexen Hüllen.

Oft wird dabei immer wieder demonstriert, dass CA-basierte Filter oft ein besseres Ergebnis als herkömmliche Filter bieten [5] [6] [7]. Dementsprechend sollen in dieser Arbeit vier CA-basierten Rauschfilter analysiert und mit einer Standard-Implementierung des Medianfilter sowie untereinander verglichen werden.

Hierfür wird zunächst die grundlegende Thematik von Bildrauschen und Filtertechniken abgehandelt. Daraufhin soll die Theorie von CA grundlegend beleuchtet werden, bevor in die spezifische Anwendung und Implementierung von CA-basierten Noise Filtern übergegangen wird.

Die Betrachtung und Analyse der Filter findet im Kontext von ausschließlich Graubildern statt. Zudem soll hier angemerkt werden, dass die hier präsentierte Bandbreite an CA-basierten Filtern nicht für die Gesamtheit aller Werke bezüglich diesen Forschungsbereich steht und lediglich als Einstiegspunkt in die Thematik sowie als Ansatzpunkt für weiterführende Forschung dienen soll.

## II. GRUNDLEGENDE THEORIE

### A. Arten von Bildrauschen

Bei Rauschen in Graubildern unterscheidet man zwischen zwei Kategorien; bei additivem Rauschen handelt es sich um eine Änderung des Signals unabhängig von dem Grundwert der Pixel [8, p. 4654], während bei multiplikativem Rauschen sich das Rauschen stattdessen proportional zur Stärke des Pixelsignals auf das Bild auswirkt [8, p. 4654].

Desweiteren gibt es verschiedene Arten von Rauschen, welche jeweils ein charakteristisches Erscheinungsbild abgeben. Im Folgenden werden vier Subtypen erläutert.

Salt-and-Pepper Rauschen (SPN, engl.: *salt and pepper noise*) beeinflusst vereinzelte Pixel und überschreibt deren Wert mit dem möglichen Minimum (0) oder Maximum (255), was zu zufällig verteilten weißen (Salt) und schwarzen (Pepper) Pixeln führt [9, p. 317]. SPN wird ausgelöst durch beispielsweise dunkle Lichtverhältnisse bei der Aufnahme des Bildes oder Übertragungsfehler bei Datentransfer [7, p. 3].

Gaußsches Rauschen folgt, gemäß seinem Namen, einer Gaußschen Normalverteilung und beeinflusst, anders als SPN, das gesamte Bild anstatt nur einzelnen Pixeln [1]. Das bewirkt, abhängig von der Varianz des Rauschens bei der jeweiligen Bildregion, extreme oder weniger starke Änderung der Pixelwerte, was zu einem weichgezeichnetem Erscheinungsbild führt [1].

Poisson Rauschen hingegen tritt meist im medizinischen Bereich auf [1]. Dieser Rauschtyp steht in Abhängigkeit zu der Helligkeit des Bildes, sodass helle Bereiche weniger und dunkle Bereiche mehr Rauschen aufweisen [1]. Das Rauschen verfolgt einer Poisson-Verteilung [1].

Der vierte Subtyp stellt das sogenannte Speckle Rauschen dar und wird durch seine körnige Textur charakterisiert [1]. Diese Art tritt ebenfalls im medizinischen Bereich bei bildgebenden Diagnostikverfahren auf [1].

Bildrauschen tritt aus verschiedensten Gründen auf und hat meist einen Informationsverlust oder -verfälschung zur Folge, was besonders im Bereich der Medizin und der Datenverarbeitung bei maschinellem Lernen zum Verhängnis wird.

#### B. Rauschfilter - Mittelwert- und Medianfilter

Im Folgenden werden zwei dieser Filter erläutert, welche oft als Absprungpunkt zur Entwicklung neuer Algorithmen dienen oder als grundlegenden Vergleich herangezogen werden. Das grundlegende Ziel bei jeglichen Filterverfahren ist meist die Reduktion des Bildrauschens bzw. Restauration des Bilds. Da es keine Ideallösung bei der Unterdrückung von Rauschen gibt, findet meist ein Abtausch zwischen der Rauschreduktion und dem Detailerhalt innerhalb des Bilds.

Der Mean Filter, auch Mittelwertfilter genannt, betrachtet jeden Pixel eines Bilds und berechnet ausgehend von seinen Nachbarn und inklusive seiner Selbst den Durchschnittswert aller Pixelsignale [9]. Die Nachbarschaft besteht hierbei aus einem Fenster (engl.: *window*) zentriert um den betrachteten Pixel [9, p. 322] – beispielsweise ein 3x3 oder 5x5 Fenster [10], wobei dieses aber keine quadratische Form einnehmen muss [9, p. 322].

Daraufhin wird der Wert des betrachteten Pixels – im weiteren Text auch Zentrum genannt – durch den Durchschnittswert ersetzt [9, p. 322]. Das hat zur Folge, dass das gesamte Bild weichgezeichnet wird und an Kantendefinition verliert, was eine Verlust von Bildklarheit zur Folge hat [10]. Das Bild erscheint insgesamt verschwommener und unschärfer als zuvor [9, p. 322].

Der Filter hat eine lineare Zeitkomplexität von  $O(N)$  mit  $N$  Bildpixel [11, p. 112], aber findet in seiner Grundform aufgrund seiner unterdurchschnittlichen Leistung wenig Verwendung in der Praxis. Jedoch stellt er ein grundlegendes Fundament für weiterführende Algorithmen und Methoden dar.

Der Medianfilter basiert auf einem ähnlichen Prinzip und gehört zu der Klasse der Rangordnungsfilter. Er ersetzt den Wert des betrachteten Pixels nicht mit dem Durchschnitt, sondern mit dem Median aller im Fenster enthaltenen Pixel [12].

Hierfür werden die Pixelwerte numerisch absteigend angeordnet und der Wert des mittleren Pixels in dieser Anordnung stellt den Median dar [12]. Im Falle einer geraden Anzahl an betrachteten Pixeln wird der Durchschnittswert der mittleren zwei Pixel genutzt [12].

Im Gegensatz zum Mittelwertfilter wird hier Bildklarheit und Definition der Kanten weitaus besser erhalten [12]. Der Filter erfindet keine neuen Pixelwerte zur Restauration, sondern nutzt schon vorhandene Signale, die im Bild auch wirklich auftreten [12] und erhöht deren Ähnlichkeit zueinander.

Dementsprechend ist der Medianfilter besonders effektiv gegen SPN, indem er korrupte Pixel ausfiltert und überschreibt [9]. Er scheitert jedoch bei hoher Rauschdichte, da es nicht genügend saubere Pixel gibt, die als Richtwert zur Restauration genutzt werden können.

Um dem etwas entgegenzuwirken, kann das Betrachtungsfenster vergrößert werden, was jedoch zu höherem Detailverlust führt. Zudem trägt diese Methode einem weiteren grundlegenden Problem bei, denn der Medianfilter weist aufgrund der Sortierung der Nachbarnpixel eine Zeitkomplexität von  $O(N * r^2 \log r)$  [13, p. 2389] auf, wobei  $N$  erneut die Pixelanzahl und  $r$  die betrachtete Fenstergröße repräsentieren. Dementsprechend skaliert die Laufzeit des Filters direkt mit der Größe des Bildes und der Größe des Fensters.

Was außerdem beiden Filtern als Hindernis zugrunde liegt, ist der Ansatz, alle Pixel gleichermaßen abzuhandeln und zu ersetzen, ungeachtet, ob diese vom Bildrauschen betroffen sind oder nicht.

Dennoch wird insbesondere der Medianfilter meist als Maßstab von effektiver SPN-Bereinigung bei niedrigem Bildrauschen genutzt und wurde auch bei den meisten hier betrachteten Werken in der internen Evaluation zum Vergleich herangezogen.

#### C. Zelluläre Automaten

Ein zellulärer Automat (CA) beschreibt einen Raum an Zellen, welcher klassischerweise ein- oder zweidimensional ist, jedoch ist eine höhere Dimensionalität ebenfalls möglich [14]. Im Rahmen dieser Arbeit wird bei einer zweidimensionalen Darstellung von einem Gitterraum ausgegangen (vgl. Abb. 1 links).

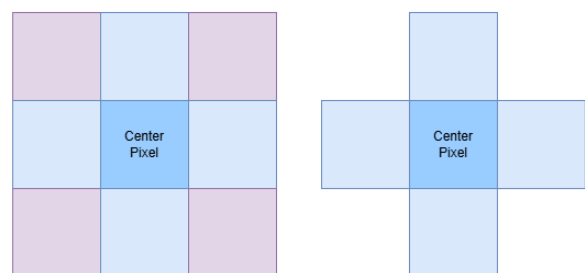


Abb. 1: Moore (links) und Von Neumann (rechts) Nachbarschaft, basiert auf [7]

Jede Zelle nimmt zum Zeitpunkt  $t$  einen Zustand  $q$  aus einer finiten Zustandsmenge  $Q$  an. Bei elementaren CA ist diese Menge beispielsweise als  $Q = \{0, 1\}$  definiert. Im Kontext von Graubildern hingegen, wäre  $Q = \{0, \dots, 255\}$  [14].

CAs gehen iterativ vor und produzieren pro Zeitschritt eine neue Generationen an Zellen [14]. Dabei hat jede Zelle eine Nachbarschaft. Bei der Betrachtung der Nachbarschaften in einem zweidimensionalen CA werden meist die sogenannten Von Neumann oder Moore Nachbarschaften in Betracht gezogen [7].

Hierbei definiert Von Neumann grundlegend alle Zellen mit einfacher Manhattan Distanz als Nachbarzellen (vgl. Abb. 1) [15, p. 27], während Moore alle anliegenden Zellen – grundlegend sind hier die acht umschließenden Zellen gemeint – als solche definiert (vgl. Abb. 2) [15, p. 27]. Diese Distanz ist jedoch variabel. Einige wissenschaftliche Arbeiten, welche in folgenden Kapiteln aufgeführt werden, nehmen größere Nachbarschaftsdistanzen wahr.

Der Neuwert der betrachteten Zelle zum Zeitpunkt  $t+1$  wird durch eine Übergangsfunktion  $f$  definiert, welche ausgehend von den Zellenzuständen in der Nachbarschaft aus der vorhergehenden Generation  $t$  einen Ausgabewert berechnet [14].

Beim Übergang von der momentanen Generation an Zellen ( $t$ ) in die nächste Generation ( $t+1$ ) definieren die jeweiligen Zellzustände in  $N$  aus der vorhergehenden Generation gemäß einer Übergangsfunktion den neuen Wert der betrachteten Zelle [14]. Die Definition von  $f$  ist hierbei willkürlich und Teil der Betrachtung und Zweckfindung von CA [14]. Zum Beispiel könnte eine Übergangsfunktion, wie folgt, lauten:

„Gibt es in der Nachbarschaft mindestens fünf lebendige Zellen ( $q = 1$ ), so ist die betrachtete Zelle ebenfalls lebendig ( $q = 1$ ), ansonsten ist sie tot ( $q = 0$ ).“

Hierbei ist anzumerken, dass die Evolution aller Zellen zu  $t+1$  parallel und unabhängig voneinander stattfinden [14].

Trotz des recht simplen lokalen Aufbaus von CA tritt auf globaler Sicht überraschend komplexes und dynamisches Verhalten hervor. Dementsprechend findet CA Anwendung in der Darstellung von dynamischen Systemen wie beispielsweise das Verhalten eines Schwarms oder die Simulation eines Waldbrandes [14].

Darüberhinaus eignet sich CA grundsätzlich zur Anwendung im Bereich der Bildverarbeitung. Die Darstellung eines Bildes als ein zweidimensionales CA ist intuitiv und naheliegend, wobei die Übergangsfunktionen zu Wertneuberechnung des Zentrums dienen. Analog könnten der Mittelwert- und Medianfilter als zweidimensionale CA betrachtet werden, welche nur eine Generation voranschreiten. Die Mittel- und Medianwertberechnungen sind hierbei die Übergangsfunktionen.

Zuletzt bedienen CAs eine weite Bandbreite an möglicher Rechenkomplexität, welche abhängig von den jeweils definierten Übergangsfunktionen ist. Hierbei behält sie zugleich einen niedrigen Rechenaufwand von  $O(T*N)$  bei, wobei  $N$  die Anzahl aller Pixel ist und  $T$  die Anzahl der Generationsdurchläufe.

#### D. Evaluationsmetrik

Um die Effektivität von Entrauschungsverfahren evaluieren zu können, gibt es eine Reihe an Methoden, über die das Endergebnis in Relation zum Originalbild bewertet werden kann.

*Peak Signal-to-Noise Ratio* (PSNR) ist definiert als

$$PSNR = 10 * \log_{10} \left( \frac{MAX^2}{MSE} \right) \quad (1)$$

wobei  $MAX$  den möglichen Maximalwert eines Pixels im Bild und  $MSE$  (engl.: *mean squared error*) die mittlere quadratische Abweichung zwischen dem Originalbild und dem restaurierten Bild bezeichnet [16].  $MSE$  ist hierbei definiert als

$$MSE = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n [I(i,j) - K(i,j)]^2 \quad (2)$$

mit  $I(i, j)$  in Referenz zum Originalbild und  $K(i,j)$  für das restaurierte Bild [16].

PSNR wird in Dezibel (dB) angegeben. Ein PSNR von unter 20 dB beschreibt ein Bild mit signifikantem Bildrauschen und -verzerrungen, über 30 dB ein Bild mit guter Qualität und über 40 dB eins, das nahezu identisch zum Originalbild ist [16] [7, p. 12].

Hierbei gilt als Faustregel, dass die Halbierung oder Verdopplung der Fehler im Bild zu jeweils einer Addition oder Subtraktion von 3dB führt, was durch den logarithmischen Aufbau dieser Metrik begründet ist [17].

Diese Metrik ist jedoch rein mathematisch und auf den numerischen Werten der Bildpixel fundiert und ignoriert die Eindrucks- und Wahrnehmung und -verarbeitung des menschlichen Auges.

Dementsprechend gibt es den *Structural Similarity Index* (SSIM) von [18], welcher folgendermaßen definiert ist:

$$SSIM(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (3)$$

$\mu_x$  und  $\mu_y$  definiert die jeweiligen durchschnittlichen Bildintensität des restaurierten ( $x$ ) und des Originalbilds ( $y$ ), während  $\sigma_x$  und  $\sigma_y$  dessen Varianzen darstellen [19].

Die  $C$ -Konstanten tragen zur Stabilisierung der Division im Falle eines sehr kleinen Divisors, der nahezu null ist, bei [19]. Der Ergebniswert liegt dann zwischen -1 und 1 [19]. Je näher der Wert an der 1 ist, desto ähnlicher ist die Struktur des restaurierten Bilds dem Originalbild [19].

Dabei sind Werte über 0.95 maßgebend für ein Bild mit Verzerrungen, welche kaum mit dem menschlichen Auge zu erkennen sind [19]. Ein SSIM von 0.80 bis 0.95 spricht von sehr guter bis nahezu hervorragender Bildqualität, während ein Bild bis 0.50 SSIM zunehmend bemerkbare Verzerrungen aufweist [19]. Bildqualitäten unter 0.50 haben schwere strukturelle Verzerrungen [19].

Der SSIM berücksichtigt bei seiner Berechnung strukturelle Information wie Luminanz, Kontrast und Textur, welche maßgebend für die Wahrnehmung des Menschen sind, und stellt dadurch eine bessere Repräsentation der Ähnlichkeit zweier Bilder dar [7].

Beim späteren Vergleich der CA-basierten Verfahren werden beide Metriken zum Vergleich herangezogen.

### III. CA-BASIERTE RAUSCHFILTER

In diesem Abschnitt werden vier CA-basierte Rauschfilter abgehandelt, welche teils gegensätzliche und teils sehr ähnliche Vorgehensweisen aufzeigen. Dies soll demonstrieren, dass auch kleine Änderungen in der Anwendung von CA großen Effekt auf die Leistung der jeweiligen Filter haben kann.

Hierbei ist anzumerken, dass jegliche Testwerte und Vergleichswerte der Algorithmen, wenn möglich, aus den jeweiligen Werken entnommen wurde, da eine praktische Implementierung und Auswertung den Umfang dieser Arbeit übersteigen.

Es wurden allgemein die PSNR und SSIM-Werte des Medianfilters aus [7] einheitlich für alle individuellen Evaluationen genutzt. Bei dieser Implementation des Medianfilters handelt es sich um den Standard mit einer Fenstergröße von  $3 \times 3$ , d.h. essenziell unter Anwendung der Moore Nachbarschaft mit einfacher Distanz. Aus naheliegendem Grund wird diese Version des Medianfilters an zutreffenden Stellen als Median M1 bezeichnet. In [7] wurde der Filter für Werte von 10% bis 90% Rauschdichte getestet, was sich bei den Testergebnissen der anderen Algorithmen widerspiegelt und sich dementsprechend für einen Vergleich darbietet.

Ausnahme von diesem Vergleichsschema stellt das Verfahren von Jeelani und Qadir [6] dar, denn innerhalb deren Tests wurden Werte von 5% bis 95% Rauschdichte abgedeckt.

Darüberhinaus wurde zur Bildrestauration überwiegend Information aus einer Moore Nachbarschaft mit doppelter Distanz genutzt, weshalb zum Vergleich Testwerte eines Medianfilter mit gleicher Nachbarschaft aus [6] entnommen wurden. Der Filter wird analog zum Median M1 bei nötiger Differenzierung Median M2 genannt.

Zuletzt wurde in [5] nur bis 80% Rauschdichte gemessen, weshalb bei dieser Evaluation ein verkürzter Graph aufzufinden ist.

Alle Daten beziehen sich auf das Lena Testbild.

#### A. Verfahren 1: Reduzierte Betrachtung der Nachbarschaft

Der Algorithmus von Kong und Choi (KCA) [7] ist das aktuellste Werk, welches in dieser Ausarbeitung betrachtet wird. Das Verfahren beschränkt sich auf eine Herangehensweise mit niedriger Komplexität, die ohne maschinelles Lernen auskommt, um eine Verwendung in ressourcenbeschränkten Umgebungen zu gewährleisten [7, pp. 2-3].

Anders als beim Mittelwert- und Medianfilter unterscheiden Kong und Choi [7] zwischen verrauschten und nicht-verrauschten Pixeln, indem deren Werte gegen 0 (Salt) und 255 (Pepper) abgeglichen und dementsprechend klassifiziert werden.

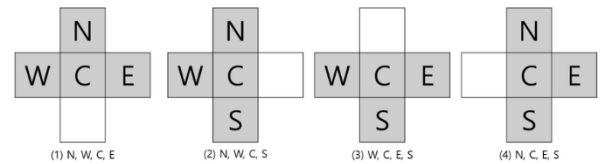


Abb. 2: Unternachbarschaften 1 bis 4, übernommen aus [7, p. 7]

Für die Neuwertberechnung vom betrachteten Pixel wird die Von Neumann Nachbarschaft genutzt [7]. Anstatt diese in ihrer Vollständigkeit in die Betrachtung miteinzubeziehen, wird sie stattdessen in vier Unternachbarschaften  $G_1$  bis  $G_4$  eingeteilt, wo jeweils ein Pixel in Himmelsrichtung exkludiert wird (siehe Abb. 2) [7].

Die Anwendung dieser Unternachbarschaften findet stets in der gleichen Reihenfolge statt, um ein konsistentes Filterverhalten über das ganze Bild hinweg sicherzustellen, ohne dabei unnötige Komplexität einzuführen [7, p. 7].

Die Autoren begründen diese statische Vorgehensweise und Definition – im Gegensatz zu einem adaptiven Verfahren wie [20] im hierauf folgenden Kapitel – außerdem damit, dass es vor allem in Bereichen, die besonders stark verrauscht oder reich an Textur sind, eine stabile und vorhersehbare Leistung gewährleistet [7, p. 8].

Der Algorithmus läuft über die Nachbarschaftsgruppen, bis eine Anzahl an nicht-verrauschten Pixeln von mindestens zwei gefunden wurde [7]. Daraufhin wird der Durchschnitt aller nicht-verrauschten Pixel als neuen Wert für den betrachteten Pixel genommen [7]. Unterschreiten alle Untergruppen diese Grenze, so wird der Pixel nicht verändert und das Verfahren läuft weiter über die restlichen verrauschten Pixel im Bild [7].

Das Verfahren geht in der Gesamtumsetzung iterativ vor [7]. In subsequenten Durchläufen reduziert sich die Rauschdichte, sodass zuvor ignorierte Pixel, deren Nachbarschaft die Reinheitsgrenze unterschritten haben, nun wiederhergestellt werden können [7]. Der Algorithmus läuft in  $O(N)$  bei einfacher Ausführung, wobei  $N$  die Gesamtanzahl aller Pixel im Bild sind [7].

#### KCA Verfahren aus [7]

```

1: Input: Noisy image  $I$ 
2: Output: Denoised image  $I'$ 
3: Identify noisy pixels  $x_i$  in  $I$  such that  $x_i = 0$  or  $255$ 
4: Repeat
5:   for each noisy pixel do
6:     for each neighbor group  $G_k$  in order ( $G_1 \rightarrow G_2 \rightarrow G_3 \rightarrow G_4$ ) do
7:       Count the number of non-noisy pixels  $n$  in  $G_k$ 
8:       if  $2 \leq n$  then
9:         Replace  $x_i$  with the mean of non-noisy pixels in  $G_k$ 
10:      break
11:     end if
12:   end for
13: end for
14: until no noisy pixels remain

```

#### 1) Vergleich mit Median M1

Im grundlegenden Vergleich mit dem Medianfilter zeigt der KCA überlegene Effektivität bei jeglicher Rauschdichte (vgl. Abb. 4Abb. 5). Vor allem in Bezug auf SSIM erreicht

das Verfahren konsistente und strukturell hochwertige Ergebnisse bis ungefähr 60% Rauschdichte bei einem SSIM von über 0.90 und degradiert erst zwischen 70% und 80% merklich.

In Bezug auf den PSNR fällt der Median schon ab 50% unter 20dB, was von signifikantem Bildrauschen spricht. Im Gegenzug behält der KCA bis einschließlich 90% ein Wert von über 20dB bei, wobei gemäß der Qualitätsgrenzen die Unterschreitungen für jeweils 40dB und 30dB zwischen 10 und 20%, und ab ungefähr 70% Rauschdichte stattfinden. Das heißt, dass der KCA auch für besonders hohe Rauschdichte sehr gute Ergebnisse erzielt.

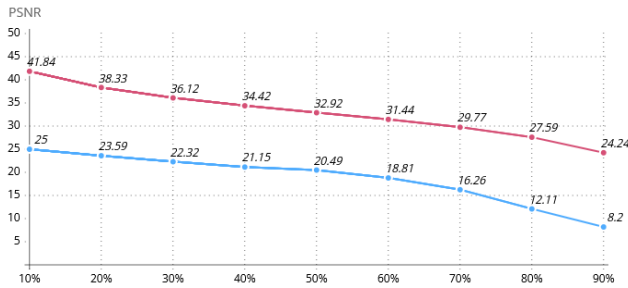


Abb. 3: PSNR-Werte für KCA (rot) im Vergleich zu Median M1 (blau), Quelle: [7]

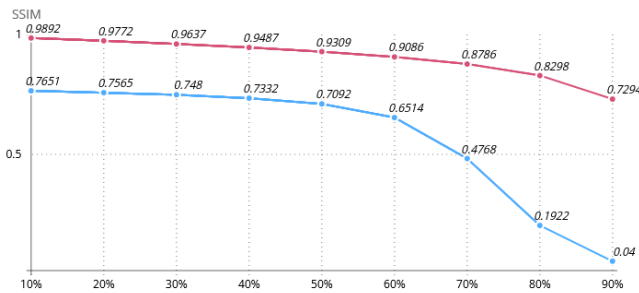


Abb. 4: SSIM-Werte für KCA (rot) im Vergleich zu Median M1 (blau), Quelle: [7]

Ein Problem, welches auch bei späteren Algorithmen übersehen wird, ist die vage Definition der benötigten Anwendungsdurchläufe des Verfahren. Zwar wird in [7] eine allgemeine Wertetabelle für die nötigen Iterationen bereitgestellt, jedoch ist diese in Bezug auf der Restauration des Lena Testbilds erstellt worden, weshalb die Allgemeingültigkeit für andere Testbilder infrage steht [vgl. 7, S. 7].

In Hinsicht auf CA-Konzepte nutzt der KCA einfache Übergangsfunktionen, um eine iterative und kleinschrittige Restauration zu bewirken. Das erlaubt dem Algorithmus eine enge Verkettung von benachbarten Pixeln bei der Neuwertberechnung, sodass nur umliegende Nachbarn maßgebend für das Zentrum sind, was den Erhalt von Kanten und Bilddetails unterstützt.

## B. Verfahren 2: Adaptive Betrachtung der Nachbarschaft

In [20] wird ein Algorithmus vorgestellt, der die Adaptivität, welche im vorherigen Werk verworfen wurde, wieder aufgreift. Angulo-Sogamoso, Gil-Sierra und Espitia-Cuchango [20] streben ein Verfahren an, welches effizient in der Entfernung von sehr hoher Rauschdichte ist, ohne dabei

rechnerische Kosten signifikant erhöhen zu müssen. Das Verfahren wird im weiteren Text, wie im originalen Werk vorgegeben, als MERIG Algorithmus bezeichnet.

Ähnlich wie beim KCA werden im MERIG Algorithmus verrauschte Pixel über einen Extremwertabgleich identifiziert [20]. Daraufhin evaluiert die Methode die Pixel, welche innerhalb der Moore Nachbarschaft mit einfacher Distanz enthalten sind, wobei das Zentrum exkludiert wird [20].

### MERIG Algorithmus (gemäß der Beschreibung in [20])

```

1: Input: Noisy image  $I$  and noise level  $n$ 
2: Output: Denoised image  $I'$ 
3: Identify noisy pixels  $x$  in  $I$  such that  $x = 0$  or 255
4: Repeat
5:   for each noisy pixel  $x$  do
6:     count number of non-noisy pixels  $n$  in neighborhood  $N$ 
9:     if  $n == 0$  and  $d \neq 4$  then
10:      increase neighborhood size  $d$  to 4 and go to 6
12:    else
13:      replace  $x_i$  with the mean of non-noisy pixels in  $N$ 
14:      break
15:    end if
16:  end for
17: end for
18: until iteration count =  $\frac{n}{10} + 1$ 

```

Sollte alle darin enthaltenen Pixel ebenfalls verrauscht sein, so wird dessen Weite zu einer Moore Nachbarschaft mit zweifacher Distanz erhöht, sodass die umgebenen Pixel  $N = 24$  evaluiert werden [20]. Zuletzt wird der Durchschnitt aller nicht-verrauschten Pixel genommen und dem Zentrum zugewiesen [20].

Da eine vollständige Restauration bei hoher Rauschdichte nach einem Durchlauf nicht möglich ist, geht der MERIG Algorithmus ebenfalls iterativ vor [20]. Anders als beim KCA wird jedoch eine genaue Anzahl an Iteration vorgegeben, welche durchgeführt werden müssen. Hierfür werden  $\frac{n}{10} + 1$  Durchläufe definiert [20, p. 3293], wobei  $n$  sich auf das Rauschlevel (engl.: *noise level*) im Bild bezieht.

Prinzipiell würde dieses Verfahren ebenfalls in  $O(N)$  bei einfacher Durchführung laufen.

### 1) Vergleich mit Median M1

In Bezug auf den Richtwert der Anzahl an Iterationen ist bedauerlicherweise unklar, ob es sich um die prozentuale Rauschdichte handelt und inwiefern diese berechnet werden sollte. Die Determinierung von Rauschdichte in einem Bild stellt ein weiteren Forschungsbereich dar, der diesen Algorithmus kostentechnisch erheblich beeinträchtigen würde.

Kanna, Bat, Vijayalakshmi und Raj [21] analysieren beispielsweise die Änderung der Entropie in verrauschten Bildern und fanden eine inverse Korrelation zwischen Rauschdichte und der Bild-Entropie – je höher die Dichte, desto niedriger wird die Entropie. Dieses Verfahren würde einen Faktor von mindestens  $O(N)$  zu dem vorherigen Aufwand hinzufügen.

Li, Yuan, Yang und Hu [22] hingegen nutzen den Korrelationskoeffizient zwischen benachbarten Pixeln eines verrauschten Bildes und berechnet die Standardabweichung von den daraus entstehenden Histogrammen im Vergleich zu einem Standard-Histogramm, welches ein nicht-verraushtes

Bild widerspiegelt. Daraufhin geben sie mittels einer Polynom-berechnung eine Schätzung über die Rauschdichte eines Bildes ab. Dieses Verfahren leistet eine vergleichsweise genaue Schätzung der prozentualen Rauschdichte [22] und würde ebenfalls einen Faktor von  $O(N)$  dem MERIG Algorithmus hinzufügen.

Desweiteren haben beider dieser Verfahren zudem das Problem, dass sie lediglich bis 70% Rauschdichte zuverlässig Ergebnisse liefern. Bei höheren Dichten degradiert entweder die Genauigkeit der Aussage [22] oder das Verfahren wurde für solch hohe Zahlen nicht getestet [21].

Insgesamt addiert die Feststellung der Rauschdichte enormen Rechenaufwand und weist auf einen möglichen Denkfehler beim Aufstellen des Verfahrens, welches zum Testen selbst Rauschen hinzufügt und klare Werte für die Rauschdichten weiß.

In Hinsicht auf CA hat MERIG einen ähnlichen Aufbau wie KCA, der dennoch leicht abweicht, was sich leistungstechnisch auf die Metriken auswirkt, wie im allgemeinen Vergleich zwischen allen Algorithmen später zu sehen sein wird.

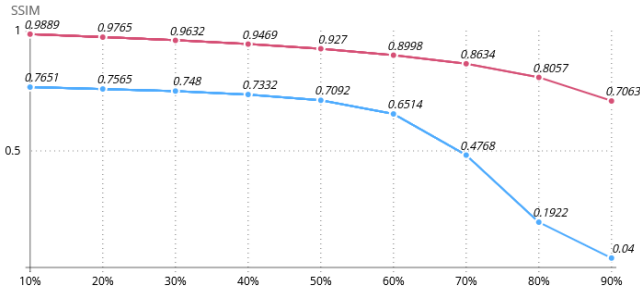


Abb. 5: SSIM-Werte für MERIG (rot) im Vergleich zu Median M1 (blau), Quelle: [1] [7]

### C. Verfahren 3: Zelluläre Automaten mit Fuzzy Logic

Im Gegensatz zu den anderen Arbeiten, die hier vorgestellt werden, nutzen Sadeghi, Rezvanian und Kamrani [5] in ihrem Algorithmus (SRKA) eine komplexere Berechnung hinsichtlich der Erkennung von verrauschten Pixeln. Hierfür wird der Durchschnittswert durch die Wurzel der genormten Summe aller Standardabweichungen der Moore-Nachbarnpixel geteilt (4) [5].

$$F(i, j) = \frac{N_m}{N_s} \quad (4)$$

$$N_m = \frac{1}{n} \sum_{i=1}^n X_i \quad (5)$$

$$N_s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (D_i - \bar{D})^2} \quad (6)$$

$$D_i = |x_{center} - x_i|, \quad 1 \leq i \leq 8 \quad (7)$$

$N_m$  ist der Durchschnitt der Werte aller Nachbarnpixel und wird berechnet über (5) [5].  $N_s$  ist die Standardabweichung aller Nachbarnpixel in Relation zum Zentrum und ist

definiert über (6), wobei  $D_i$  die Standardabweichung eines Nachbarnpixels zum Zentrum ist (7) [5].  $\bar{D}$  ist der Durchschnitt von allen Abweichungen  $D_i$  [5].

Der Wert  $F(i, j)$  wird dann mit einem zuvordefiniertem Grenzwert  $\theta$  verglichen und ausgehend davon als verrauscht oder sauber klassifiziert, falls dieser über- oder unterschritten wird [5].

Wurde ein korrupter Pixel identifiziert, so geht der SRKA in die zweite Phase über - Restauration durch Fuzzy-CA (FCA) [5].

Fuzzy-Logik, auch Unschärfemodellierung genannt, wandelt exakte Messwerte - z.B. 0 und 1 - in graduelle Zugehörigkeitswerte - z.B. zwischen 0 und 1 - um [23]. Dies stellt eine repräsentativere Darstellung von Verhältnissen in der echten Welt dar und verhilft zu besser fundierten Entscheidungen [23].

Fuzzy-Logik besteht aus mehreren Phasen [23], aber in Kontext von diesem Algorithmus sind nur Fuzzifizierung und Defuzzifizierung von Relevanz [5, p. 777]. Beim Fuzzifizieren werden exakte, numerische Werte in eine Fuzzy-Menge durch eine geeignete Zugehörigkeitsfunktion überführt [23]. Um die Werte dann wieder scharf zu stellen, werden die Fuzzy-Daten durch Defuzzifizierung in den originalen Wertebereich übergeführt [23].

SRKA nutzt beim Restaurationsschritt die Von Neumann Nachbarschaft und berechnet für jeden Nachbar-Pixel zunächst dessen Fuzzy-Darstellung (8) [5, p. 777]. Es wird für  $w$  der Wert 127.5 angesetzt.

$$M_i(x) = \frac{|w - x_i|}{w}, \quad 0 \leq x \leq 255, w = 127.5 \quad (8)$$

Beim Defuzzifizieren wird dann der durchschnittliche Schwerpunkt der Nachbarschaft berechnet und dem Zentrum als Neuwert zugewiesen (9) [5].

$$Y = \frac{\sum_{i=1}^n M_i * x_i}{\sum_{i=1}^n M_i} \quad (9)$$

Dieser Algorithmus wird wiederholt angewendet, bis die Differenz der Änderung der Pixelwerte in subsequenten Durchläufen einen gewissen Grenzwert unterschreitet und somit einen stabilen Zustand erreicht [5, p. 777].

#### SRKA Verfahren aus [5]

- 
- 1: **Input:** Noisy image  $I$
  - 2: **Output:** Denoised image  $I'$
  - 3: Identify  $x_{i,j}$  in  $I$  such that  $F(i, j) \geq \theta$
  - 4: **Repeat**
  - 5:     **for** each noisy pixel  $x_{i,j}$  **do**
  - 6:         **for** each pixel  $y_i$  in Von Neumann neighborhood **do**
  - 7:             fuzzify  $y_i = M_i(y)$
  - 8:         **end for**
  - 9:         replace  $x_{i,j}$  with average center of gravity  $Y$
  - 10:     **end for**
  - 11: **until** changes in pixels converge to stable state
-

### 1) Vergleich mit Median M1

Dieses Verfahren zeigt eine erhöhte Komplexität bei der Identifikation von verrauschten Pixeln auf. Jedoch wird die Definition eines verrauschten Pixels im Gegensatz zu den anderen Algorithmen in Korrelation zu den Werten seiner Nachbarspixel betrachtet und erlaubt dadurch eine differenziertere Identifikation von SPN im Kontext seiner Umgebung.

Weiterhin ist aber das Ausmaß dieser Identifikationsmethode sowie dessen Effektivität fragwürdig. Sadeghi, Rezvanian und Kamrani [5] geben keinen Richtwert für den Grenzwert  $\theta$  an und dokumentierten auch ihre Testwerte nicht in ausreichendem Detail, sodass dieser aus deren Bericht entnommen werden konnte.

Desweiteren wurde von den Autoren angemerkt, dass verrauschte Pixel durch dieses Verfahren nur ungefähr erkannt werden können [5, p. 776]. Das ist im Kontext von SPN, welches Extremwert-Rauschen definiert, unzureichend ist, da ein simpler Wertevergleich zur Identifikation ebenfalls ausreicht, wie es die anderen Algorithmen [7] und [20] bewiesen haben.

Hier stellt sich die Frage, ob eine vereinfachte Identifikationsmethode eine ähnliche Leistung erreichen würde wie die dargestellte Version.

SRKA gibt ebenfalls eine Definition an, wie viele Anwendungsdurchläufe für eine erfolgreiche Restauration benötigt werden. Jedoch wird dieser Aspekt, wie bei MERIG auch, bei der Dokumentation stark vernachlässigt. Infolgedessen ist unklar, wie der Grenzwert definiert wird und was akzeptable Werte darstellt.

Ungeachtet dessen, zeigt auch der SRKA eine klare Verbesserung in Bezug auf Leistung und Ergebnisqualität. Es behält bis ungefähr 45% Rauschdichte gute Bildqualität bei mit einer PSNR von über 30dB (vgl. Abb. 6).

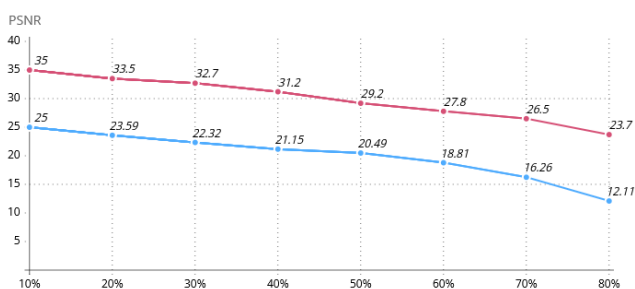


Abb. 6: PSNR-Werte für SRKA (rot) im Vergleich zu Median M1 (blau), Quelle: [5] [7]

Hier macht sich ein Trend bei dem Aufbau der CA-basierten Filtern bemerkbar, denn die Struktur vom SRKA ist dem KCA und MERIG - und wie später auch bei Algorithmus von [6] zu sehen sein wird – nicht unähnlich. Die Grundstruktur – die Identifikation des verrauschten Pixels über den Zustand der Nachbarschaft, die Restauration des Zentrums durch eine Übergangsfunktion, sowie die iterative Vorgehensweise – ist in allen Verfahren gleich und lediglich die Übergangsfunktionen unterscheiden sich.

Das ist eine triviale Erkenntnis, jedoch wird später ersichtlich, dass ausgehend von den verwendeten Übergangsfunktionen, sich die Leistung der jeweiligen Verfahren teils drastisch unterscheiden.

### D. Verfahren 4: Direktes Ersetzen des Zentrums

Der letzte Algorithmus, der zum Vergleich herangeführt werden soll, wurde von Jeelani und Qadir [6] konzipiert und wird zunächst in vier Versionen präsentiert, welche alle dem gleichen Grundprinzip folgen.

Die Identifikation von verrauschten Pixeln folgt dem bekannten Verfahren und wird durch einen Extremwertabgleich – 0 für Salt, 255 für Pepper - festgemacht [6].

|             |             |             |             |             |
|-------------|-------------|-------------|-------------|-------------|
| $\Psi_{21}$ | $\Psi_{13}$ | $\Psi_9$    | $\Psi_{14}$ | $\Psi_{22}$ |
| $\Psi_{15}$ | $\Psi_5$    | $\Psi_1$    | $\Psi_6$    | $\Psi_{16}$ |
| $\Psi_{10}$ | $\Psi_2$    | $C(i,j)$    | $\Psi_3$    | $\Psi_{11}$ |
| $\Psi_{17}$ | $\Psi_7$    | $\Psi_4$    | $\Psi_8$    | $\Psi_{18}$ |
| $\Psi_{23}$ | $\Psi_{19}$ | $\Psi_{12}$ | $\Psi_{20}$ | $\Psi_{24}$ |

Abb. 7: Primärregeln  $\Psi_1$  bis  $\Psi_{24}$  gemäß Zentrumsnähe, übernommen aus [6, p. 367]

Konzeptionell betrachtet der Algorithmus eine Moore Nachbarschaft mit zweifacher Distanz [6]. Die Nachbarspixel werden als sogenannte Primärregel (engl.: *primary rule*) von  $\Psi_1$  bis  $\Psi_{24}$  basierend auf deren Nähe zum Zentrum durchnummeriert und stellen später den Ersatzwert des Zentrums dar (siehe Abb. 7) [6, p. 367]. Die Durchnummerierung der Pixel ist hierbei nicht unähnlich einer Von Neumann Betrachtung der Nachbarschaft.

In der ersten Version des Filters (Filter 1) werden Primärregeln 1 bis 12 genutzt [6, p. 367]. Bei Anwendung iteriert der Filter über das gesamte Bild, bis ein verrauschter Pixel aufgefunden wurde, woraufhin die erste Primärregel in Sequenz gewählt wird, welche nicht verrauscht ist [6]. Die Restauration des Pixels findet unter Ersetzen des Zentrums über den Wert der gewählten Regel statt [6, p. 367].

Während der Testphase wurden vertikale Folgefehler im Bild aufgefunden, welche durch das Abarbeiten in normaler Abfolge bedingt waren [6]. Dementsprechend wurde eine zweite Iteration (Filter 2) konzipiert, welche das Bild in umgekehrter horizontaler und vertikaler Reihenfolge abgearbeitet – von unten nach oben und rechts nach links [6, p. 367].

Diese Änderung konnte die Folgefehler etwas eindämmen, aber nicht vollständig eliminieren [6]. Darauf folgende Iterationen verliefen wieder in gleicher Abfolge jedoch mit anderen Sets an Primärregeln [6].

Dieses Verfahren geht ebenfalls iterativ vor [6].

Um die Länge dieser Arbeit einzugrenzen, wird zu Vergleichszwecken repräsentativ Filter 2 aus den vorgeschlagenen Verfahren gewählt, da es sich Metrik-basiert um die effektivste Version handelt [6, p. 373]. Dieser Filter wird analog zu den anderen Verfahren im weiteren Text mit JQA abgekürzt.

#### JQA Verfahren aus [6]

```

1:  Input: Noisy image  $C$ 
2:  Output: Denoised image  $C'$ 
3:  for all pixel  $(i, j)$  in  $C$  in reverse order do
4:    if  $C(i, j) = 0$  or  $255$  then
5:      for  $k \leftarrow 1$  to  $12$  do
6:        if  $\psi_k(C(i, j)) \neq 0$  and  $255$  then
7:           $C(i, j) = \psi_k(C(i, j))$ 
8:        break
9:      end if
10:   end for
11: end if
12: end for

```

#### 1) Vergleich mit Median M2

Der JQA stellt innerhalb der hier präsentierten Verfahren das rechnerisch einfachste Verfahren dar, denn der Zielwert des Zentrums wird nicht nochmals über die Nachbarschaft berechnet und stattdessen direkt über die gewählte Primärregel restauriert.

Um einen Vergleich zu einem Median mit ähnlicher genutzter Informationsdichte zu gewährleisten, wurde eine Version des Filters genutzt, welche ebenfalls eine Von Neumann Nachbarschaft mit zweifacher Distanz nutzt. Trotz seiner Einfachheit übertrifft auch dieser Algorithmus den Medianfilter und weist hohe Effektivität vor.

Trotz seiner Einfachheit übertrifft auch dieser Algorithmus den Medianfilter und weist hohe Effektivität vor. In Relation zum PSNR fällt die Qualität des JQA jedoch schon ab 35% unter 30dB, was sich auch im SSIM widerspiegelt, aber liefert akzeptable Ergebnisse bis einschließlich 65% (vgl. Abb. 8, 9). Ab dieser Rauschdichte treten jedoch stark bemerkbare visuelle Verzerrungen auf, welche sich über schlieren-ähnlich Artefakte äußern. Diese ziehen sich tendenziell von dem Zentrum-Pixel in Richtung der gewählten Primärregel über das Bild.

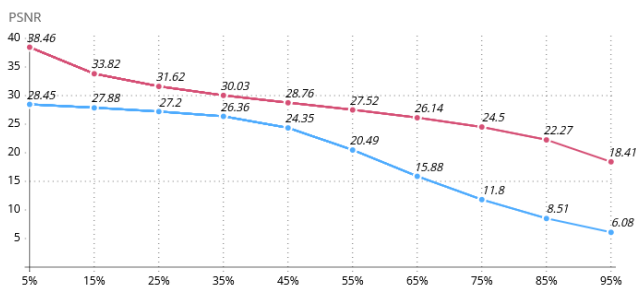


Abb. 8: PSNR-Werte für JQA (rot) im Vergleich zu Median M2 (blau), Quelle: [6]

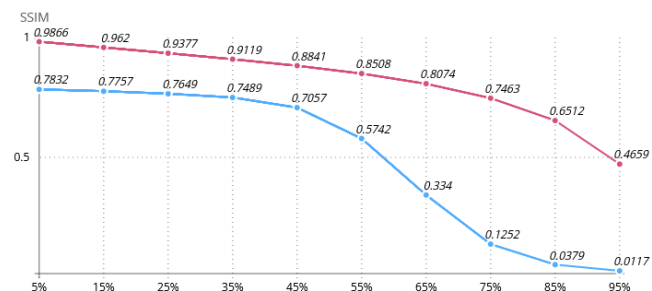


Abb. 9: SSIM-Werte für JQA (rot) im Vergleich zu Median M2 (blau), Quelle: [6]

#### IV. VERGLEICH UND ANALYSE ALLER ALGORITHMEN

Wie zuvor erwähnt, nutzen Jeelani und Qadir [6] einen Testwertbereich von 5% bis 95% Rauschdichte. Obwohl die Effektivität des gegebenen Algorithmus stetig abnimmt und ein plötzlicher Anstieg in den Zwischenintervallen unwahrscheinlich ist, sind die Zwischenwerte in den Graphen dennoch interpoliert und nicht maßgebend für die wirklich Leistung des JQA bei den jeweiligen Rauschdichten.

Desweiteren zeigt die Effektivität der Standard-Implementation des Medianfilters teils große Abweichungen zwischen verschiedenen wissenschaftlichen Werken [7] [6], welche nicht zu vernachlässigen sind. Zwischen den Testdaten aus [7] und [6] herrschen beispielsweise Diskrepanzen von bis zu 4dB PSNR und 0.10 SSIM bei deren Median M1 Implementierungen, was beim PSNR eine signifikante Differenz darstellt. Infolgedessen sind allgemeine Aussagen von doppelter Effektivität und dergleichen möglicherweise nicht allgemeingültig zutreffend.

Dessen ungeachtet, erfolgt dennoch ein allgemeiner Vergleich der Algorithmen in Hinsicht auf PSNR und SSIM. Vergleichswerte zum Median M1 und M2 werden jeweils aus [7] und [6] genommen. Im Falle vom SRKA und MERIG Algorithmus finden diese nur in Hinsicht auf deren jeweilige getestete Metrik – je PSNR und SSIM – statt.

Für die Evaluation wird beim PSNR mathematisch bedingt eine Differenz von 3dB als signifikant angesehen [17], während beim SSIM eine Differenz von 0.05 als bedeutend eingestuft wird, wobei Grenzüberschreitung nochmals aussagekräftiger sind [16] [19].

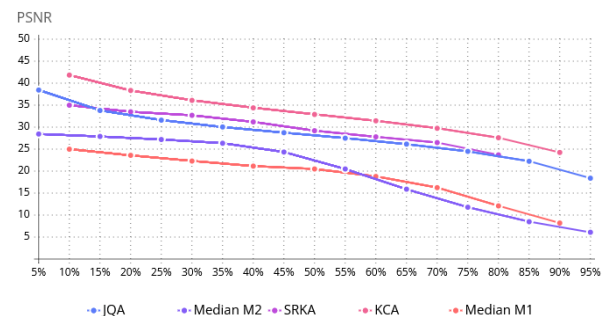


Abb. 10: PSNR-Werte aller Algorithmen im Vergleich zu Median M1 und M2, Quelle: [7] [20] [6] [5]

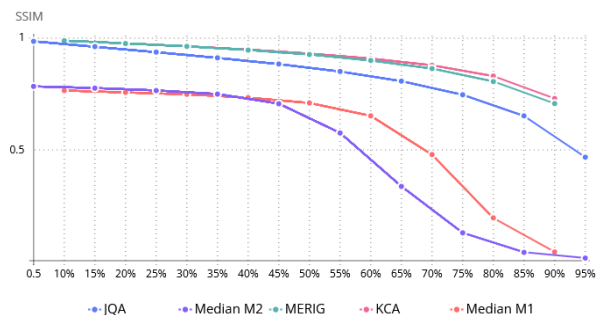


Abb. 11: SSIM-Werte aller Algorithmen im Vergleich zu Median M1 und M2, Quelle: [7] [20] [6] [5]

Wie schon in den individuellen Evaluationen aus vorhergehenden Sektionen angemerkt, übertreffen jegliche hier präsentierte Algorithmen sowohl den Median M1 als auch den

Median M2 und zeigen erhebliche Verbesserung in Bildqualität bezüglich PSNR und SSIM nach Restauration (vgl. Abb. 10, 11).

Aus Abbildung 11 ist zu entnehmen, dass JQA und SRKA eine ähnliche Leistung über alle getesteten Rauschdichten erreichen, wobei SRKA jedoch bei mittleren Rauschdichten zwischen ungefähr 30% und 40% Rauschdichte visuell eine minimale Überlegenheit zeigt (vgl. Abb. 10), aber keine signifikante Änderung im PSNR-Wert aufweist (vgl. Tab. 1).

Dieses Ergebnis ist insofern signifikant, dass die beiden Algorithmen einen gegenteiligen Ansatz hinsichtlich der Neuberechnung des Zentrums beziehen. Bei JQA wird lediglich ein nicht-verrauschter Nachbarpixel genommen, während SRKA über die entfernungsbasierten Zugehörigkeiten der Nachbarschaft das Zentrum neu kalkuliert.

Im Gegensatz dazu drückt KCA eine nahezu konstante Differenz von über 3dB im Vergleich zu MERIG und SRKA aus (vgl. Tab. 1), was im Umkehrschluss auf eine Halbierung der Fehler im Ergebnisbild deutet [17]. Dieser Unterschied zwischen KCA und JQA wird auch hinsichtlich der SSIM-Metrik belegt; JQA fällt schon ab 25% Rauschdichte unter einem Wert von 0.95, während der KCA bis fast 40% über dieser Grenze bleibt (vgl. Abb. 11, Tab. 1).

Dieser PSNR-Effektivitätsdifferenz bleibt auch bei höheren Rauschdichten erhalten, sodass JQA, im Vergleich zu

KCA, meist schon 15% früher bestimmte Qualitätsgrenzen unterschreitet (vgl. Tab. 1).

Jedoch hat vergleichsweise MERIG eine ähnliche Leistung wie KCA mit nahezu identischen SSIM-Werten bis mindestens 30% Rauschdichte (vgl. Abb. 11). Daraufhin divergieren die beiden Algorithmen mit einer Differenz von annähernd 0.03.

Auch hier geht die ähnliche Leistung dieser beiden Algorithmen gegen angenommene Erwartungen, da die genutzte Informationsbreite der beiden Algorithmen sich erheblich unterscheiden und erneut einen Gegensatz darstellen.

KCA benutzt ausschließlich eine reduzierte Von Neumann Nachbarschaft mit einfacher Distanz, während MERIG adaptiv zwischen Moore mit einfacher oder doppelter Distanz wechselt, was eine unterschiedliche Informationsbreite von 3:8 bzw. 3:24 (KCA: MERIG) darstellt. Diese haben jedoch keine Auswirkung auf die Leistung.

Dies ist intuitiv durch die iterative Anwendung der Algorithmen zu begründen, wodurch KCA in subsequenten Schritten Restaurationen aus vorherigen Durchläufen nachholen kann. Darüberhinaus begrenzt sich KCA mehr auf die direkten Pixelnachbarn, wodurch der Erhalt von Kanten und Bilddetail unterstützt wird, was die Differenz in Effektivität erklären könnte.

TABELLE I PSNR-WERTE FÜR KCA, SRKA, JQA, MEDIAN M1 UND M2

| PSNR | Rauschdichte |       |       |       |       |       |       |       |       |       |       |       |       |       |      |       |       |       |       |
|------|--------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|------|-------|-------|-------|-------|
|      | 5%           | 10%   | 15%   | 20%   | 25%   | 30%   | 35%   | 40%   | 45%   | 50%   | 55%   | 60%   | 65%   | 70%   | 75%  | 80%   | 85%   | 90%   | 95%   |
| M1   |              | 25    |       | 23.59 |       | 22.32 |       | 21.15 |       | 20.49 |       | 18.81 |       | 16.26 |      | 12.11 |       | 8.2   |       |
| M2   | 28.45        |       | 27.88 |       | 27.2  |       | 26.36 |       | 24.35 |       | 20.49 |       | 15.88 |       | 11.8 |       | 8.51  |       | 6.08  |
| KCA  |              | 41.84 |       | 38.33 |       | 36.12 |       | 34.42 |       | 32.92 |       | 31.44 |       | 29.77 |      | 27.59 |       | 24.24 |       |
| SRKA |              | 35    |       | 33.5  |       | 32.7  |       | 31.2  |       | 29.2  |       | 27.8  |       | 26.5  |      | 23.7  |       |       |       |
| JQA  | 38.46        |       | 33.82 |       | 31.62 |       | 30.03 |       | 28.76 |       | 27.52 |       | 26.14 |       | 24.5 |       | 22.27 |       | 18.41 |

TABELLE II SSIM-WERTE FÜR KCA, SRKA, JQA, MEDIAN M1 UND M2

| SSIM         | Rauschdichte |        |        |        |        |        |        |        |        |        |        |        |        |        |        |        |        |        |        |
|--------------|--------------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
|              | 5%           | 10%    | 15%    | 20%    | 25%    | 30%    | 35%    | 40%    | 45%    | 50%    | 55%    | 60%    | 65%    | 70%    | 75%    | 80%    | 85%    | 90%    | 95%    |
| <i>M1</i>    |              | 0.7651 |        | 0.7565 |        | 0.748  |        | 0.7332 |        | 0.7092 |        | 0.6514 |        | 0.4768 |        | 0.1922 |        | 0.04   |        |
| <i>M2</i>    | 0.7832       |        | 0.7757 |        | 0.7649 |        | 0.7489 |        | 0.7057 |        | 0.5742 |        | 0.334  |        | 0.1252 |        | 0.0379 |        | 0.0117 |
| <i>KCA</i>   |              | 0.9892 |        | 0.9772 |        | 0.9637 |        | 0.9487 |        | 0.9309 |        | 0.9086 |        | 0.8786 |        | 0.8298 |        | 0.7294 |        |
| <i>MERIG</i> |              | 0.9889 |        | 0.9765 |        | 0.9632 |        | 0.9469 |        | 0.927  |        | 0.8998 |        | 0.8634 |        | 0.8057 |        | 0.7063 |        |
| <i>JQA</i>   | 0.9866       |        | 0.962  |        | 0.9377 |        | 0.9119 |        | 0.8841 |        | 0.8508 |        | 0.8074 |        | 0.7463 |        | 0.6512 |        | 0.4659 |

## V. FAZIT UND ANMERKUNGEN

Insgesamt veranschaulichen die hier analysierten Algorithmen, dass die Grundkonzepte von CA großes Potential für die Anwendung in der Bildverarbeitung präsentieren. Das spiegelt sich auch in den Evaluationen wider, denn alle hier dargestellten Algorithmen weisen eine ähnliche Rechenkomplexität wie der Mean Filter und zugleich eine höhere Effektivität als der Medianfilter auf.

Hierbei ist anzumerken, dass die Mean und Medianfilter konzeptionell nicht ungleich CA-basierten Filtern sind. Sie nutzen ebenfalls Nachbarschaftslogik zur Neuberechnung

des Zentrums, wobei diese Rechnungen als eine Art Übergangsfunktion angesehen werden können. Der größte und zugleich bedeutendste Unterschied liegt jedoch im iterativen Charakter von CA, der für eine ressourcen-leichte Methodik erlaubt, welche mit wenig Information pro Iteration auskommt, wie der Vergleich zwischen KCA und MERIG bewiesen hat.

Grundlegend herrschen zwischen den Algorithmen bezüglich der Herangehensweise und Einbindung von CA keine besonders großen Unterschiede. Identifikation von SPN findet meist über Extremwertvergleiche statt, während Zentrum-Pixel meist durch den Durchschnittswert der sauberen Nach-

barapixel ersetzt wird. Dennoch gibt es signifikante Unterschiede in der Leistung der Algorithmen, was charakteristisch für CA und emergentes Verhalten ist; kleine Änderung in den Übergangsfunktionen führen zu enormen Änderungen im finalen Verhalten des CAs, was weiterführende Forschung in diesem Bereich zulässt und fördert.

Anschließend ist hier anzumerken, dass der Korpus an Werken in diesem Forschungsbereich eine klare Unterrepräsentation von Verarbeitung farblicher Bildern sowie Behandlung von anderen Arten von Rauschen aufweist.

Abgesehen von [24], welches einen Filter für Speckle Rauschen entwickelt hat, wurde in jeglichen Verfahren fast ausschließlich SPN abgehandelt, weshalb weiterführende Forschungen in diese Unterbereiche empfehlenswert wären, um die Eignung von CA auch bei anderen Rauscharten zu belegen.

## REFERENCES

- [1] Z. Ansari, "Understanding Image Noise: Exploring Different Kinds of Noises in Image Processing," 4 February 2025. [Online]. Available: [https://medium.com/@zar\\_373/understanding-image-noise-exploring-different-kinds-of-noises-in-image-processing-dcd26c6c9e5a](https://medium.com/@zar_373/understanding-image-noise-exploring-different-kinds-of-noises-in-image-processing-dcd26c6c9e5a).
- [2] A. Popovici and D. Popovici, "Cellular Automata in Image Processing," in *Fifteenth International Symposium on Mathematical Theory of Networks and Systems*, 2002.
- [3] P. L. Rosin, "Training cellular automata for image processing," *IEEE Transactions on Image Processing*, vol. 15, no. 7, pp. 2076-2087, 2006.
- [4] P. L. Rosin, "Image processing using 3-state cellular automata," *Computer Vision and Image Understanding*, vol. 114, no. 7, pp. 790-802, 2010.
- [5] S. Sadeghi, A. Rezvanian and E. Kamrani, "An efficient method for impulse noise reduction from images using fuzzy cellular automata," *AEU - International Journal of Electronics and Communications*, vol. 66, no. 9, pp. 772-779, 2012.
- [6] Z. Jeelani and F. Qadir, "Cellular automata-based approach for salt-and-pepper noise filtration," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 2, pp. 365-374, December 2018.
- [7] G. Kong and U. Choi, "A cellular automata-based method for salt-and-pepper noise removal in images," *Multimedia Tools and Applications*, vol. 85, no. 3, 2026.
- [8] N. Chandra, S. Teotia, A. Chaudhary, V. Jain, T. Vashishtha and N. N. Dwivedi, "Noise Reduction Using Cellular Automata," *NeuroQuantology*, vol. 20, no. 6, pp. 4650-4672, 2022.
- [9] R. C. Gonzales and R. E. Woods, "Restoration in the Presence of Noise Only - Spatial Filtering," in *Digital Image Processing*, New Jersey, Prentice Hall, 2008, pp. 322-335.
- [10] University of Edinburgh, "Spatial Filters - Mean Filter," [Online]. Available: <https://homepages.inf.ed.ac.uk/rbf/HIPR2/mean.htm>. [Accessed 24 June 2026].
- [11] O. F. Syah, R. Amar, D. S. Dzaqi, M. Adiputra, F. Zukman and M. G. Efgivia, "Real-Time Feasibility vs Frequency Analysis," *International Journal of Scientific Engineering and Science*, vol. 9, no. 3, pp. 107-113, 2025.
- [12] University of Edinburgh, "Spatial Filters - Median Filter," 2020. [Online]. Available: <https://homepages.inf.ed.ac.uk/rbf/HIPR2/median.htm>. [Accessed 24 June 2026].
- [13] S. Perreault and P. Hebert, "Median Filtering in Constant Time," *IEEE Transactions on Image Processing*, vol. 16, no. 9, pp. 2389-2394, 2007.
- [14] D. Shiffman, "7. Cellular Automata," 2024. [Online]. Available: <https://natureofcode.com/cellular-automata/>. [Accessed 21 June 2026].
- [15] K. P. Haderler and J. Müller, "Cellular Automata: Basic Definitions," in *Cellular Automata: Analysis and Applications*, Springer International Publishing, 2017, pp. 19-35.
- [16] H. Lakhloufi, "Peak Signal-to-Noise Ratio (PSNR)," MetricGate, 18 October 2025. [Online]. Available: <https://metricgate.com/docs/peak-signal-to-noise-ratio/>. [Accessed 26 June 2026].
- [17] N. Sapunov, "PSNR Explained: What It Measures and Where It Lies · Telemedicine · Fora Soft Learn," Fora Soft, 23 June 2026. [Online]. Available: <https://www.forasoft.com/learn/video-quality/articles-vqm/psnr-explained>. [Accessed 1 July 2026].
- [18] Z. Wang, A. Bovik, H. Sheikh and E. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600-612, 2004.
- [19] H. Lakhloufi, "Structural Similarity Index (SSIM)," MetricGate, 10 February 2026. [Online]. Available: <https://metricgate.com/docs/structural-similarity-index/>. [Accessed 26 June 2026].
- [20] K. Angulo-Sogamoso, D. Gil-Sierra and H. Espitia-Cuchango, "A NOVEL ALGORITHM BASED ON CELLULAR AUTOMATA TO ELIMINATE NOISE IN DIGITAL IMAGES," *ARPN Journal of Engineering and Applied Sciences*, vol. 14, no. 18, pp. 3289-3300, 2019.
- [21] R. Kanna B, M. Shafi Bhat, C. Vijayalakshmi and A. N. J. Raj4, "Model to estimate the salt and pepper noise density level on gray-scale digital image," *EAI Endorsed Transactions on Energy Web*, vol. 5, no. 18, June 2018.
- [22] T. Y. Li, P. G. Yuan, D. Y. Yang and X. Y. Hu, "Estimation of Salt-Pepper Noise in Images with Correlation Inspection," *Applied Mechanics and Materials*, Vols. 462-463, pp. 391-398, 2013.
- [23] S. Priy, "Fuzzy Logic | Introduction - GeeksforGeeks," 4 October 2018. [Online]. Available: <https://www.geeksforgeeks.org/artificial-intelligence/fuzzy-logic-introduction/>. [Accessed 16 June 2026].
- [24] S. A. Twinkle, S. Kamilya and J. Mukherjee, "Despeckling Images Using Elementary Cellular Automata," *Communications in Computer and Information Science*, vol. 2499, pp. 191-202, 2025.
- [25] "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600-612, April 2004.
- [26] A. Mazraeh, "Week 3: Fuzzification and Defuzzification," 11 February 2025. [Online]. Available: <https://medium.com/@adnan.mazraeh1993/fuzzification-and-defuzzification-6bfe059047e>. [Accessed 27 June 2026].